

PAPER • OPEN ACCESS

Quantum versus classical generative modelling in finance

To cite this article: Brian Coyle *et al* 2021 *Quantum Sci. Technol.* **6** 024013

View the [article online](#) for updates and enhancements.

You may also like

- [Classical versus quantum models in machine learning: insights from a finance application](#)
Javier Alcazar, Vicente Leyton-Ortega and Alejandro Perdomo-Ortiz
- [Tensor tree learns hidden relational structures in data to construct generative models](#)
Kenji Harada, Tsuyoshi Okubo and Naoki Kawashima
- [Bayesian learning of parameterised quantum circuits](#)
Samuel Duffield, Marcello Benedetti and Matthias Rosenkranz

Unlock Quantum Computing Potential with Multiphysics Simulation

Design the infrastructure that enables quantum performance.

The future of computing depends on stable, scalable quantum technologies capable of solving problems beyond the reach of classical supercomputers.

COMSOL Multiphysics® empowers engineers to simulate and optimise the supporting systems behind quantum platforms — including RF and microwave components, photonics and optics, cryogenic cooling and thermal management. By modelling the coupled physics that shape the qubit environment, you can reduce noise, improve stability and accelerate quantum hardware development.

» comsol.com/industry/electronics/quantum-computing

Quantum Science and Technology



PAPER

Quantum versus classical generative modelling in finance

OPEN ACCESS

RECEIVED
30 July 2020

REVISED
15 October 2020

ACCEPTED FOR PUBLICATION
15 December 2020

PUBLISHED
12 April 2021

Original content from
this work may be used
under the terms of the
[Creative Commons
Attribution 4.0 licence](#).

Any further distribution
of this work must
maintain attribution to
the author(s) and the
title of the work, journal
citation and DOI.



Brian Coyle^{1,*} , Maxwell Henderson³, Justin Chan Jin Le³, Niraj Kumar¹, Marco Paini⁴
and Elham Kashefi^{1,2}

¹ School of Informatics, 10 Crichton Street, Edinburgh EH8 9AB, United Kingdom

² CNRS, LIP6, Sorbonne Université, 4 place Jussieu, 75005 Paris, France

³ Rigetti Computing, Berkeley, CA 94710, United States of America

⁴ Rigetti Computing, London, United Kingdom

* Author to whom any correspondence should be addressed.

E-mail: dbrian.coyle@ed.ac

Keywords: generative modelling, Born machine, Boltzmann machine, finance

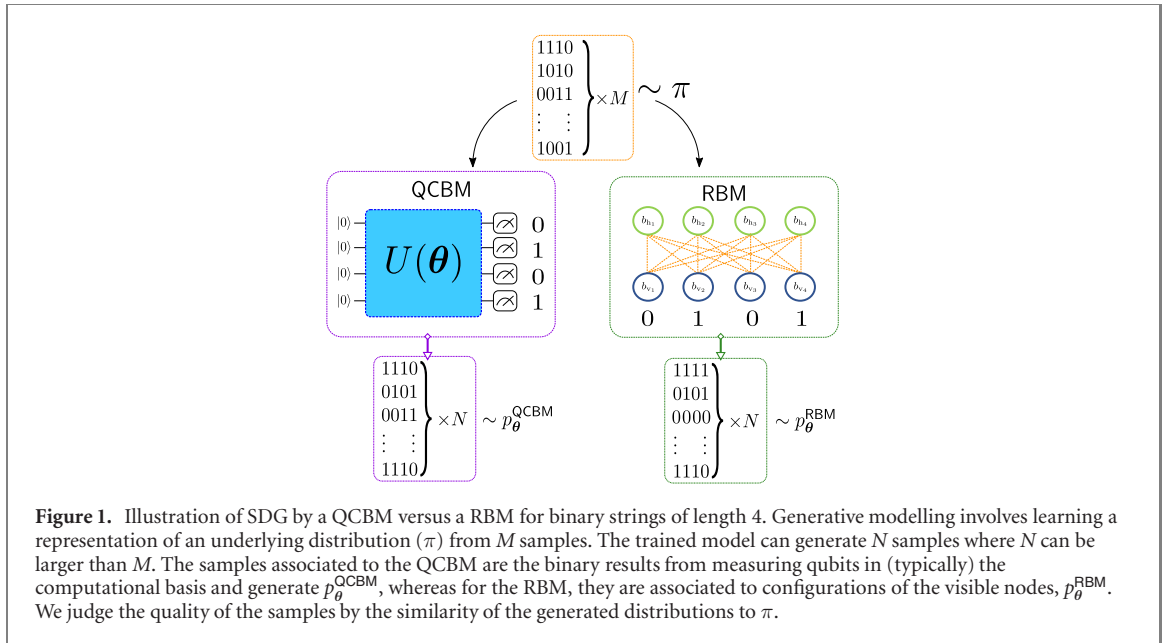
Abstract

Finding a concrete use case for quantum computers in the near term is still an open question, with machine learning typically touted as one of the first fields which will be impacted by quantum technologies. In this work, we investigate and compare the capabilities of quantum versus classical models for the task of generative modelling in machine learning. We use a real world financial dataset consisting of correlated currency pairs and compare two models in their ability to learn the resulting distribution—a restricted Boltzmann machine, and a quantum circuit Born machine. We provide extensive numerical results indicating that the simulated Born machine always at least matches the performance of the Boltzmann machine in this task, and demonstrates superior performance as the model scales. We perform experiments on both simulated and physical quantum chips using the Rigetti QCSTM platform, and also are able to partially train the largest instance to date of a quantum circuit Born machine on quantum hardware. Finally, by studying the entanglement capacity of the training Born machines, we find that entanglement typically plays a role in the problem instances which demonstrate an advantage over the Boltzmann machine.

1. Introduction

The prediction power of machine learning algorithms is limited by the quality of the datasets used to train the models. In the age of big data, possessing high-quality data can offer significant competitive advantage to institutions who utilize machine learning in their core business operations such as Facebook, Google and Amazon. However, for many organizations high-quality data can be scarce. This is because training data for industrial problems are often plagued by erroneous information, limited by privacy and over-fitting. Hence, high-quality data can be expensive or even impossible to obtain especially for machine learning applications at industrial scales. Synthetic data generation (SDG) bridges the gap for training better machine learning models when such data is not readily available. Rather than collecting raw data, SDG uses statistical methods, simulation modelling, and neural networks to generate a synthetic equivalent of the real-world data set (i.e. sample generation). SDG allows users to overcome data scarcity, avoid privacy issues, and overcome over-fitting problems at lower costs. This is achieved by SDG removing erroneous or mislabelled data, as each sample is generated from predefined parameters to produce clean and machine learning-ready datasets. SDG can also produce realistic data for unobserved scenarios to train more generalized models. In machine learning terms, SDG is typically achieved by *generative modelling* or distribution learning. Using quantum models for SDG has garnered interest due to the ease of generating data samples (alternatively, performing the ‘inference’ step) from a quantum distribution, whereas even the very act of sample generation can be difficult classically, which we elaborate on through the text.

In terms of quantum capabilities, we are now firmly in the noisy intermediate scale quantum (NISQ) [1] era, where we have access to small, error-prone quantum computers, but which are sufficiently powerful to



be able to address problems which are not classically simulatable [2]. However, finding a useful application for such devices is a non-trivial task, with quantum chemistry [3, 4] or quantum optimization [5, 6] being the usual suspects for areas in which to search. Problems in finance have also proved to be a lucrative area of study [7–10]. With each discovered use case, an argument is frequently required as to why such a problem could not have been tackled by purely classical methods. The primary approaches to gain an advantage with quantum computers study the computational time complexity in solving these problems. The claims of exponential speedups [11, 12] in these cases usually rely on the non-existence of unlikely relationships between computational complexity classes. Furthermore, when (especially) dealing with heuristic algorithms, concrete evidence of a speedup can be difficult to find, or manifests as a result of suboptimal parameter choices [13]. However, simply solving the problem faster is not the only way in which quantum computers can gain victories. Alternatively, one can examine other relevant problem dimensions, such as accuracy of solution, which is the goal we aim for in this work.

We explore two different machine learning approaches for generating synthetic financial market data. One model is completely classical (although trained using simulated quantum methods): the restricted Boltzmann machines (RBMs) and the other is completely quantum in nature: a quantum circuit Born machines (QCBMs). This is similar to other recent works [14], which addressed financial problems with these two models and found that the Born machine has the capacity to outperform the Boltzmann machine, when it comes to generating synthetic data. In this work, we draw a similar conclusion by enforcing similar constraints on both models in order to make as fair a comparison as possible (figure 1).

In section 2 we discuss the main ideas involved in generative modelling, and elaborate on the two models we use for this task. We also discuss the financial dataset we use for training. In section 3, we detail the specific architectures for the Boltzmann and Born machines, namely the underlying graph structures and the circuit ansätze for the QCBM. In section 4, we describe the training protocols we use for each model and finally in section 5 we detail the numerical results we find, and showcase examples where the Born machine outperforms the Boltzmann machine in learning the financial dataset. We present simulated and experimental results implemented on the Rigetti QPU [15] using Quantum Cloud Services (QCSTM). Finally, we conclude in section 6 and discuss future work.

2. Generative modelling

Generative models are powerful machine learning models, which essentially aim to learn a probability distribution, denoted π , over some data (say vectors, $\mathbf{x}$), which is sampled from π , $\mathbf{x} \sim \pi(\mathbf{x})$. A typical use case is in classification tasks, where a generative model seeks to learn the joint distribution over data and labels, y , $\pi(\mathbf{x}, y)$. We assume the distribution in question is defined over the space of binary strings of length n , $\{0, 1\}^n$. A generative model can be typically parameterised by some parameters, θ , and are represented by an output ‘model’ distribution over the data, which is a function of those parameters, $p_{\theta}(\mathbf{x})$. The goal of training a generative model is to force the model distribution as close as possible to the data distribution, relative to some measure. This is done by finding a suitable setting of the parameters, typically using some

optimization routine. In practice however, we typically do not have access to the true distributions (meaning their explicit probability density functions or otherwise). This is inevitably true when using implicit models⁴ like generative adversarial networks (GANs) [17] or quantum circuit distributions, which by their very nature are distributions which are not directly accessible [2] due to the classical intractability of them. In this work, we assume we have N samples from the model distribution, $\{\mathbf{x}_i\}_{i=1}^N, \mathbf{x}_i \sim p_\theta(\mathbf{x})$ and M samples from the data distribution, $\{\mathbf{y}_j\}_{j=1}^M, \mathbf{y}_j \sim \pi(\mathbf{y})$.

Common use cases for generative models are in image generation, but they have also received interest from the quantum computing community, as the acceleration in training of generative models using quantum techniques was one of the early areas of interest in the field of quantum machine learning [18]. The focus of the area has shifted somewhat in recent years, from accelerating training and inference of *classical* models using quantum techniques, to the development of *completely new* models in the quantum world. One of the earliest examples of which is the quantum Boltzmann machine (QBM), which is a generalization of the classical Boltzmann machine (see section 2.2). This was followed by the introduction of Born machines [19] and QCBMs [20, 21], which sample from the fundamentally quantum distribution underlying a pure state of a quantum system. One of the most recent additions to this family are Hamiltonian based models and the variational quantum thermalizer [22], which generalizes all of the above since it contains the distribution provided by a *mixed* quantum state as the underlying model. The latter is also a generalization of ‘energy-based’ models, of which the Boltzmann machine is an example. Furthermore, quantum generative models are some of the most promising applications for near term quantum computers since their nature aligns them closely with demonstrations of ‘*quantum supremacy*’ [2] and such connections have recently been made [23, 24] with extensions into different architectures [25].

In this work we focus on two of these models in order to make a direct comparison and study any potential indication of quantum advantage for these models over purely classical generative models. We investigate a Born machine and a RBM and make a thorough comparison between the two for a generative modelling task. We do this using a realistic dataset in a financial application, which facilitates a simple way to compare the models at differing scales. Our motivation is the work of [26] which showed the outperformance of an RBM over parametric models, for this dataset, which are the common tool used in the finance industry. This was subsequently followed by the subsequent outperformance of the RBM by a Born machine [14] on the same dataset. However, the degree to which this advantage was observable was not obvious. This research and [14] supplements the work of [27] which demonstrated a similar outperformance of an RBM by a QCBM, but for a different problem domain. Our work expands on the latter by running larger problem instances on simulators and physical hardware, using alternative training methods, and also using alternative methods of comparison of the models. Finally, drawing a comparison between a Born and Boltzmann machine is part of the goal of [19], in which they consider the problem from a mutual information point of view. They further conjecture that properties such as mutual information of the dataset, and entanglement entropy in the target problem, an/or model would be useful in determining problems where the Born machine could have superior performance over an RBM.

2.1. Born machine

A Born machine [19] is a fundamentally quantum model, which achieves SDG by generating samples according to Born’s rule of quantum mechanics. The fundamentally non-classical nature of the model has provided motivation for why it can outperform classical models in at least its expressive power [23, 24]. This expressive power translates in an ability to represent certain distributions efficiently which cannot be done by any classical model, for example, those utilized in a recent demonstration of quantum computational supremacy [2].

In the most common scenario, a binary sample, $\mathbf{x} \in \{0, 1\}^n$, is generated from a quantum state, ρ , according to:

$$\mathbf{x} \sim p(\mathbf{x}) = \text{Tr}(|\mathbf{x}\rangle\langle\mathbf{x}| \rho), \quad (1)$$

where $|\mathbf{x}\rangle\langle\mathbf{x}|$ is the projector onto the computational basis state described by $\mathbf{x}$. In order to obtain a trainable machine learning model, we parameterize the state: $\rho \rightarrow \rho_\theta$. We also further consider the scenario where the parameterised state is a pure state, i.e. $\rho_\theta := |\psi_\theta\rangle\langle\psi_\theta|$. In this case, the correlations present in the model will be of a purely quantum nature. The parameterised distribution is then:

$$\mathbf{x} \sim p_\theta^{\text{QCBM}}(\mathbf{x}) = |\langle\mathbf{x}|\psi_\theta\rangle|^2. \quad (2)$$

Finally, if the state, $|\psi_\theta\rangle$ is generated by a quantum circuit (as opposed to, for example, by a continuous time Hamiltonian evolution), the model is referred to as a QCBM [20, 21]. In this form, the ease of

⁴ These are models for which we do not have explicit access to the underlying probability density function [16].

performing inference becomes apparent: once trained, the parameterized quantum state prepared by a quantum circuit and then simply measured. The measurement results then constitute an (approximate) sample from the data distribution. Furthermore, utilizing quantum randomness as a sample generation mechanism this way relaxes the need to input randomness into the model as is usually done to build GANs. However, we mention that inputting randomness has been considered in the quantum case [28] as well, although the advantage of doing so has yet to be explored.

A generalization of the above can be achieved by relaxing the purity assumption of the underlying state, and doing so results in quantum Hamiltonian based models [22], which instead can carry *both* classical and quantum correlations.

In order to find a good fit to the data distribution, π , such that the model, $p_\theta(\mathbf{x})$, can effectively generate synthetic data, an optimization routine is invoked to search over the space of possible states $|\psi_\theta\rangle$. Since Born machines are implicit models, careful consideration must be given to the choice of optimization routine, since any optimizer must be able to effectively, and efficiently, deal with samples alone. One may consider quantum training procedures [29], but more commonly the optimization procedure will be a fully classical routine. This makes these models hybrid quantum–classical in nature and therefore friendly to NISQ devices, only using the quantum resource when necessary.

2.2. Boltzmann machine

Generalized Boltzmann machines (GBMs) are graphical models with powerful synthetic data-generation capabilities. While GBMs can vary significantly in terms of how they are applied to various problems and their particular architectures (see some example architectures in figure 2), they all share some defining characteristics. The model architecture is defined by a graph $\mathbf{G}$, which consists of a set of edges, $\mathcal{E}$, and nodes (vertices) which we denote $\mathcal{N}$. Each edge, $e \in \mathcal{E}$ has a corresponding edge *weight*, $W_e \in \mathcal{W}$. In generality, the edge weights can also be self-loops (biases in standard Boltzmann machine terminology), or hyper-edges (edges connecting more than two nodes) as illustrated in figure 2(c). Crucially, the nodes are typically partitioned into *visible* and *hidden* nodes, $\mathcal{N} = \mathcal{N}_v \cup \mathcal{N}_h$. The visible nodes directly model some aspect of the data distribution, while the hidden nodes are used for capturing features of the data, and are not tied to any particular aspect of it. As such the hidden nodes typically correspond directly to the expressive power of the model. Finally, a sample generated by the GBM is distributed according to the Boltzmann distribution:

$$\mathbf{x} \sim p_\theta^{\text{GBM}}(\mathbf{x}) = \frac{e^{-\beta E(\mathbf{x})}}{\mathcal{Z}}, \quad (3)$$

$p_\theta^{\text{GBM}}(\mathbf{x})$ is the probability to observe the visible nodes in some state $\mathbf{n}_v = \mathbf{x}$, and describes the model distribution for the Boltzmann machine. $\mathbf{n}_v$ is a particular state (corresponding to a binary vector) of the visible nodes in $\mathcal{N}_v$. In this case, the model parameters are the weights of the machine, $\theta = \mathcal{W}$. E and $\mathcal{Z}$ is the model *energy* (defining an energy based model [30]) and *partition function* respectively, and are defined by:

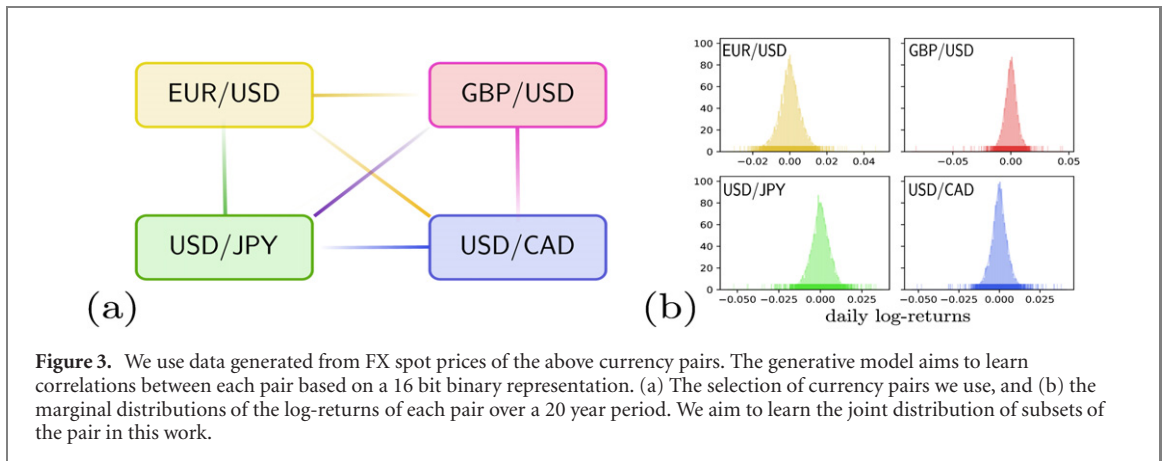
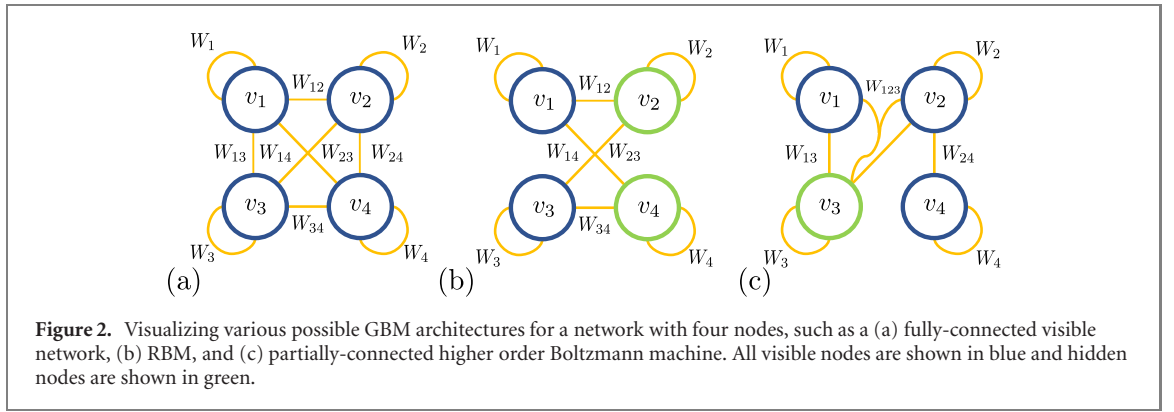
$$E(\mathbf{v}) := - \sum_{e \in \mathcal{E}} W_e \prod_{v_i \in e} v_i, \quad (4)$$

$$\mathcal{Z} := \sum_{\mathbf{v}} e^{-\beta E(\mathbf{v})}. \quad (5)$$

The notation, $v_i \in e$, refers to the nodes connected to edge e , and β is an effective inverse temperature term. The sum in equation (5) is taken over all possible binary vectors $\mathbf{v} \in \{0, 1\}^n$. In this work, we focus specifically on the *restricted* version of the Boltzmann machine (RBM) corresponding to figure 2(b), and we discuss this specification further in section 3.2. In this case, we denote $p_\theta^{\text{GBM}}(\mathbf{x}) \rightarrow p_\theta^{\text{RBM}}(\mathbf{x})$ where the latter distribution is generated by marginalizing over hidden units.

Finally, while all of the above is purely classical (in contrast to the Born machine, a GBM carries only classical correlations), the extension of the *model itself* into the quantum world has also been proposed in the QBM [31–34] as we mentioned above. In this framework, the energy function, equation (4) is replaced by a quantum Hamiltonian and the model distribution in question is generated by sampling from the thermal state of this Hamiltonian, mimicking a Boltzmann distribution. This thermal state can be prepared either by quantum annealing [31] or by a gate based approach [35]. By introducing off-diagonal terms in this Hamiltonian, non-trivial quantum behaviour can be exploited, and the model inherits some characteristics of a Born machine (i.e. some of the randomness originates from Born’s rule).

In this work, however, we focus on the GBM as a completely classical object, which we detail in section 3, however, we do leverage quantum inspired training methods which are discussed in section 4.



Furthermore, as mentioned we only study the RBM here, but we discuss the extension of the methods in this work to the more general Boltzmann machine structures in section 6.

2.3. A financial dataset

In order to perform SDG, we require some dataset to learn. In this work, we focus on one of a financial origin, in particular one considered by [26]. This dataset contains 5070 samples of daily log-returns of 4 currency pairs between 1999–2019 (see figure 3). In order to fit on the binary architecture of the Born and Boltzmann machines, the spot prices of each currency pair are converted to 16 bit binary values, resulting in samples of 64 bits long. This discretisation provides a convenient method for fitting various problem sizes onto models with different numbers of qubits or visible nodes for the Born machine or RBM respectively. In particular, we can tune both the number of currency pairs (i), and the precision of each pair (j) so the problem size is described by a tuple (i, j) . For example, as we revisit in section 3, a 12 qubit Born machine can be tasked to learn the distribution of 4 currency pairs at 3 bits of precision, 3 pairs with 4 bits or 2 pairs at 6 bits of precision.

3. Model structures

Here we provide specific details about the model architectures we choose to use, in order to derive as fair a comparison as possible. In the first instance, we choose to only train the bias terms in the RBM (the self-loops in figure 2) for simplicity. We also fix the number of parameters in the Born machine by the number of layers, and then match the number of parameters in the RBM to this, since it is simpler to grow the number of RBM parameters by simply adding extra nodes.

3.1. Born machine ansatz

The ansatz which we use for the QCBM is hardware efficient as we endeavor to run the model on real quantum hardware. We also restrict the number of parameters in the circuit to match the number used in the RBM, following [27]. We choose this hardware native approach to closely fit the structure of Rigetti's chip design (the structure of the AspenTM-7 and AspenTM-8 can be seen in figure 4). Furthermore, we solely parameterize the single qubit unitaries to avoid compilation overheads arising out of two qubit

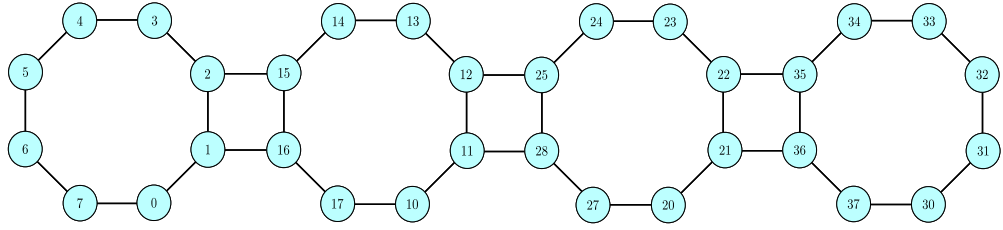


Figure 4. Aspen™-7/Aspen™-8 32 qubit chip designs. Note not all connections shown above are directly accessible on the chip itself.

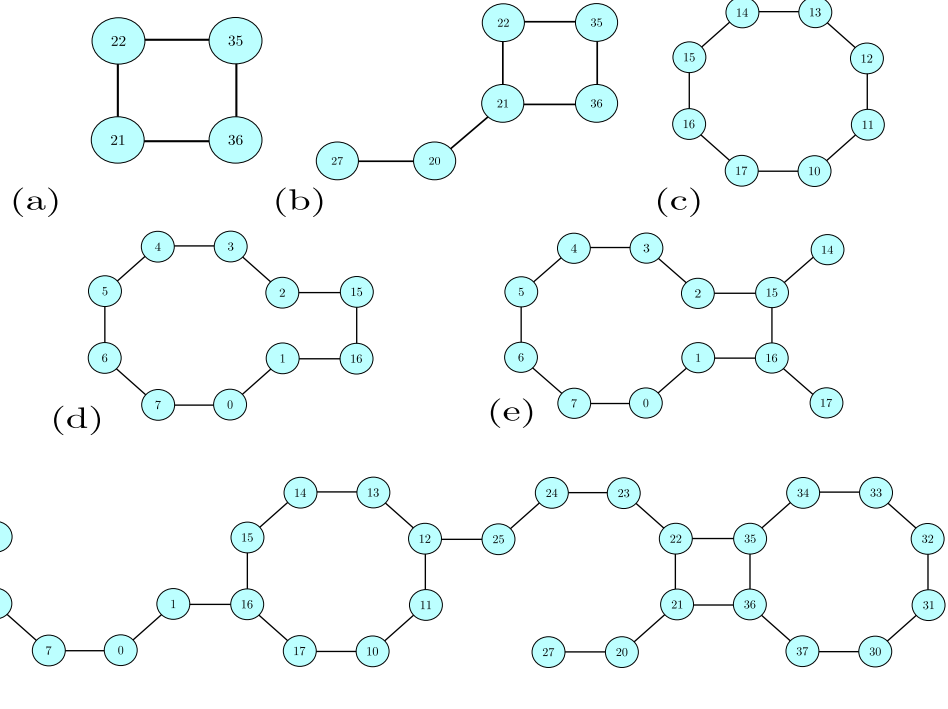


Figure 5. Select sublattices from the Aspen™-7 and Aspen™-8 chips, corresponding to different problem sizes. Figure shows the (a) Aspen™-7-4Q-C, (b) Aspen™-7-6Q-C, (c) Aspen™-7-8Q-C, (d) 10 qubit Aspen™-8, (e) 12 qubit Aspen™-8 and (f) Aspen™-7-28Q-A sublattices. Using these topologies we can fit problems of size (2, 2), (2, 3), (2, 4), (2, 5), (2, 6) and (4, 7) respectively, where the notation (i, j) indicates i currency pairs, each described by j bits of precision.

unitary parameterization. If we were to do so, we could employ a similar strategy to [36], which uses ‘blocks’ of parameterized unitaries in such a way to enforce a linear scaling of the number of parameters with the number of qubits, when building a quantum classifier.

We run all experiments using the Rigetti Aspen™-7 and Aspen™-8 chips, which are designed to contain 32 qubits, however some qubits are not available. The chips can be broken up into sublattices containing fewer qubits, some examples can be seen in figure 5. The largest sublattice on the Aspen™-7 chip is the Aspen™-7-28Q-A which contains 28 usable qubits (seen in figure 5(f)).

For each of the lattices in figure 5, we fit the native entanglement structure using CZ gates, and layers of single qubits rotations. For convenience, we use R_y rotation gates as the single qubit gates, which have the decomposition $R_y(\theta) = R_x(\pi/2)R_z(\theta)R_x(-\pi/2)$, using the Rigetti native single qubit rotations. The first ‘layer’ contains only R_y gates, and each layer thereafter consists of the hardware native CZ gates, plus a layer of R_y gates. For the 4 qubit chip, Aspen™-7-4Q-C, we illustrate this in figure 6. For the other sublattices in figure 5, we illustrate the entanglement structure in figure 7 for the first layer of the circuits. In this way, an n qubit QCBM with l layers will have $n \times l$ trainable parameters. While hardware efficient circuit ansätze, such as those we use here are subject to ‘barren plateaus’ (BPs) (see [37–41], among others), which are regions of exponentially vanishing gradients leading to circuits which are untrainable via gradient descent, we do not address this question here, primarily due to the small problem sizes we deal with. This issue is tightly related to the choice of ansatz, and ansätze which avoid BPs usually incorporate some problem information into their structure, as in [41]. It is still an open question how to do this effectively in the

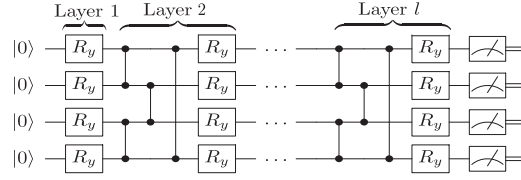


Figure 6. Hardware efficient circuit for the Aspen™-7-4Q-C, with l layers using the native entanglement structure native to the chip.

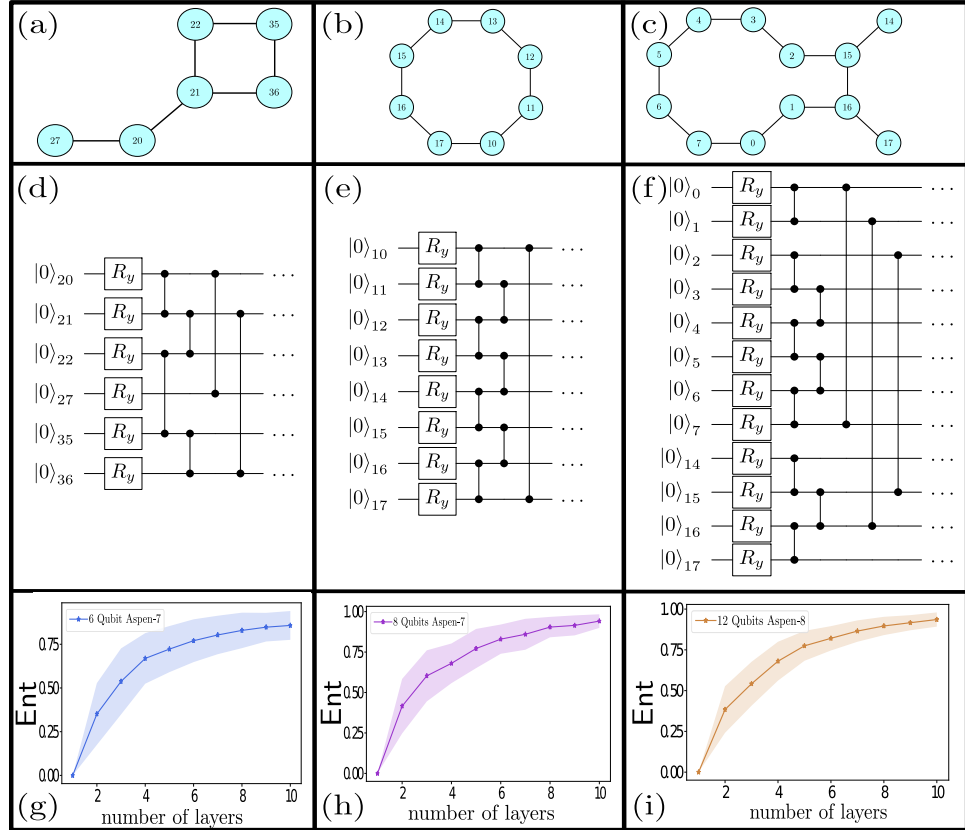


Figure 7. Hardware efficient circuits for 6, 8, 12 qubit Born machine ansatz. (a)–(c) Show Aspen™-7-6Q-C, Aspen™-7-8Q-C from the Aspen™-7 chip and a 12 qubit sublattice from the Aspen™-8 chip which we consider. (d)–(f) Illustrate the entanglement structure in a single layer, which tightly matches the chip topology. (g)–(i) Show the average entangling capability, Ent in (10) as a function of the number of layers in the circuit for each of the entangling structures shown in (d)–(f). Error bars show mean and standard deviation over 100 random parameter instances, $\{\theta_i\}_{i=1}^{100}, \theta_i^j \sim U(0, 2\pi)$, in the single qubit rotations. U is the uniform distribution over the interval $[0, 2\pi]$.

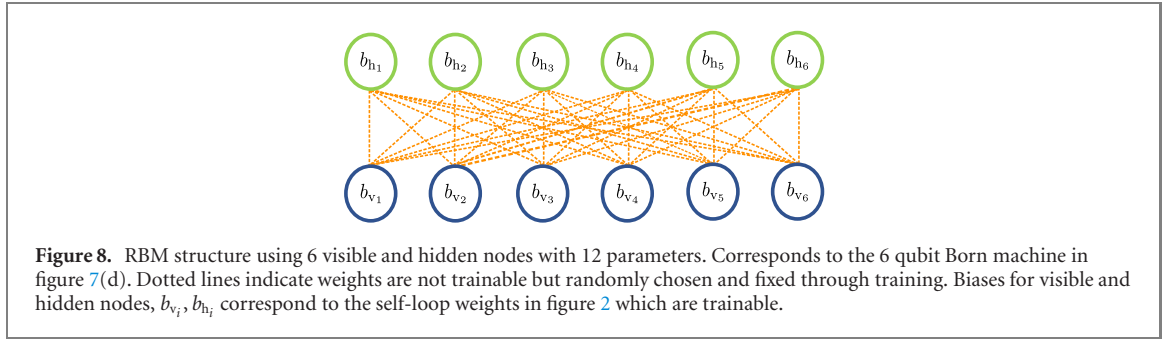
data-driven world of ML. This is an extremely relevant and interesting question which we aim to address in future work. In particular, this issue will likely become prominent as the model scales to larger problem sizes (figure 8).

For the above circuits, we compute the average Meyer–Wallach [42] entanglement capacity, a measure of entanglement in quantum states proposed as a method of comparing different circuit ansätze by [43]. This measure has been used in a similar context by [44] in order to draw connections between ansatz structure and classification accuracy. The entanglement measure Q is defined, for a given input state $|\psi\rangle$ as:

$$Q(|\psi\rangle) := \frac{4}{n} \sum_{j=1}^n D(\iota_j(0) |\psi\rangle, \iota_j(1) |\psi\rangle) \quad (6)$$

$$D(|\mathbf{u}\rangle, |\mathbf{v}\rangle) = \frac{1}{2} \sum_{i,j} |u_i v_j - u_j v_i|^2, \quad (7)$$

where D is a particular distance between two quantum states, $|\mathbf{u}\rangle := \sum_i u_i |i\rangle, |\mathbf{v}\rangle := \sum_j v_j |j\rangle$. This distance can be understood as the square of the area of the parallelogram created by vectors $|\mathbf{u}\rangle$ and $|\mathbf{v}\rangle$. The



notation $\iota_j(b)$ is a linear map which acts on computational basis states as follows:

$$\iota_j(b) |b_1 \dots b_n\rangle := \delta_{bb_j} |b_1 \dots \hat{b}_j \dots b_n\rangle, \quad (8)$$

where $\hat{\cdot}$ indicates the absence of the j th qubit. For example, $\iota_2(0) |1001\rangle = |101\rangle$. However, to evaluate Q for a quantum state, we instead use the equivalent formulation derived by [45], which involves computing the purities of each subsystem of the state $|\psi\rangle$:

$$Q(|\psi\rangle) = 2 \left(1 - \frac{1}{n} \sum_{k=1}^n \text{Tr}[\rho_k^2] \right), \quad (9)$$

where $\rho_k := \text{Tr}_k(|\psi\rangle\langle\psi|)$ is the partial trace over every one of the n subsystem of $|\psi\rangle$ *except* k . This reformulation of Q gives more efficient computation and operational meaning since the purity of a quantum state is efficiently computable. Given Q , we define [43] **Ent** as the average value of Q over a set, $\mathcal{S}$ of M randomly chosen parameter instances, $S := \{\theta_i\}_{i=1}^M$:

$$\text{Ent} := \frac{1}{|S|} \sum_i Q(|\psi_{\theta_i}\rangle). \quad (10)$$

For the circuit ansätze we choose, the value of **Ent** is plotted for a given number of layers in figure 7.

3.2. Boltzmann machine structure

Given the above choice for a Born machine ansatz, we can build a corresponding RBM which has $n_v := |\mathcal{N}_v| = n$ visible nodes (where n is the number of qubits) and $n_h := |\mathcal{N}_h| = nl - n = n \times (l - 1)$ hidden nodes. To reiterate, we fix the RBM weights to have random values and only the local biases are trained. We revisit weight training in appendix B.3.

4. Training procedures

In order to fit the model distribution to the data, one needs some means of comparing how close these two distributions are. Typically, this comes in the form of a *cost function*, $D(p_\theta, \pi)$. In this work, we consider a variety of cost functions with which to compare both models we investigate.

This cost function is then minimized during the training procedure to find a setting of the parameters, θ such that $D(p_\theta, \pi)$ is as small as possible. Gradient descent (GD) is a common method to minimize such costs in machine learning as it finds the steepest direction of descent in the parameter landscape defined by, θ . GD proceeds with a number of ‘epochs’, where in each epoch (t) the parameters are updated as follows:

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \Delta D(p_{\theta^{(t)}}, \pi) \quad (11)$$

$\Delta D(p_\theta, \pi)$ is the update rule defining how each parameter should be updated, depending on the current value of D and is negative since we wish to go downhill in the parameter landscape. The ‘vanilla’ form of gradient descent simply directly uses an update of the form $\Delta D(p_\theta, \pi) = \eta \partial_{\theta^{(t)}} D(p_\theta, \pi)$, where η is a *learning rate* and $\partial_{\theta^{(t)}} D$ is the partial derivative of D with respect to the current parameters. Computing this gradient efficiently can be a non-trivial procedure, and it is estimated given the data. More complicated update rules such as Adam [46] are also possible, which include terms like ‘momentum’ to the update rule, to improve convergence speed.

4.1. Born machine training

The primary cost function we choose to train the Born machine is the Sinkhorn divergence (SHD), a recently defined [47–49] method of distribution comparison, and related to *optimal transport* (OT) [50], which is known to be a relatively powerful metric between probability distributions.

$$D(p_\theta, \pi) := \mathcal{L}_{\text{SHD}}^\epsilon(p_\theta, \pi) := \text{OT}_\epsilon^c(p_\theta, \pi) - \frac{1}{2}\text{OT}_\epsilon^c(p_\theta, p_\theta) - \frac{1}{2}\text{OT}_\epsilon^c(\pi, \pi) \quad (12)$$

$$\text{OT}_\epsilon^c(p_\theta, \pi) := \min_{U \in \mathcal{U}(p_\theta, \pi)} \left(\sum_{\substack{(\mathbf{x}, \mathbf{y}) \\ \in \mathcal{X}^d \times \mathcal{Y}^d}} c(\mathbf{x}, \mathbf{y}) U(\mathbf{x}, \mathbf{y}) + \epsilon \text{KL}(U | p_\theta \otimes \pi) \right), \quad (13)$$

where $\epsilon \geq 0$ is a regularisation parameter, and $\mathcal{U}(p_\theta, \pi)$ is the set of all *couplings* between p_θ and π , i.e. the set of all joint distributions, whose marginals with respect to $\mathbf{x}, \mathbf{y}$ are $p_\theta(\mathbf{x}), \pi(\mathbf{y})$ respectively. $\text{KL}(U | p_\theta \otimes \pi)$ is the Kullback–Leibler [51] divergence (also relative entropy) between the coupling, U , and a product distribution composed of the model and the data, $p_\theta \otimes \pi$. The introduction of the entropy term smooths the problem, so that it becomes more easily solvable, as a function of ϵ .

We use this cost function since we numerically found it to be the best choice, in terms of speed and accuracy of training. However, we provide a comparison to the *maximum mean discrepancy* (MMD) cost function, training with respect to an adversarial discriminator and a gradient free genetic algorithm in appendix A.

As shown in [24] we can derive gradients of the SHD, with respect to the given parameter, θ_k , since each parameterised gate we employ has the form $U(\theta) = \exp(\frac{i\theta\Sigma}{2})$, where $\Sigma^2 = 1$. Using the parameter shift rule [52, 53], the gradient can be written as follows:

$$\frac{\partial \mathcal{L}_{\text{SHD}}^\epsilon(p_\theta, \pi)}{\partial \theta_k} = \sum_{\mathbf{x}} \frac{\partial \mathcal{L}_{\text{SHD}}^\epsilon(p_\theta, \pi)}{\partial p_\theta(\mathbf{x})} \frac{\partial p_\theta(\mathbf{x})}{\partial \theta_k} \quad (14)$$

$$= \frac{1}{2} \sum_{\mathbf{x}} \varphi(\mathbf{x}) \left(p_{\theta_k^+}(\mathbf{x}) - p_{\theta_k^-}(\mathbf{x}) \right) \quad (15)$$

$$= \frac{1}{2} \left(\mathbb{E}_{\mathbf{x} \sim p_{\theta_k^+}} [\varphi(\mathbf{x})] - \mathbb{E}_{\mathbf{x} \sim p_{\theta_k^-}} [\varphi(\mathbf{x})] \right). \quad (16)$$

The function φ is defined [49] in order to ensure the gradient extends to the entire sample space, and is defined as follows for each sample, $\mathbf{x}$:

$$\begin{aligned} \varphi(\mathbf{x}) = & -\epsilon \text{LSE}_{k=1}^M \left(\log \left(\pi(\mathbf{y}^k) + \frac{1}{\epsilon} g(\mathbf{y}^k) - \frac{1}{\epsilon} C(\mathbf{x}, \mathbf{y}^k) \right) \right) \\ & + \epsilon \text{LSE}_{k=1}^N \left(\log \left(p_\theta(\mathbf{x}^k) + \frac{1}{\epsilon} s(\mathbf{x}^k) - \frac{1}{\epsilon} C(\mathbf{x}, \mathbf{x}^k) \right) \right). \end{aligned} \quad (17)$$

Therefore, one can compute the gradient by drawing samples from the distributions, $\hat{\mathbf{x}} \sim p_{\theta^\pm}$, and computing the vector $\varphi(\mathbf{x})$, for each sample, $\mathbf{x}$. The functions g and s in equation (17) are optimal Sinkhorn potentials, arising from a primal-dual formulation of OT. These are computed using the Sinkhorn algorithm, which gives the divergence its name [54]. $C(\mathbf{x}, \mathbf{y})$ is the OT *cost matrix* derived from the cost function applied to all samples, $C_{ij}(\mathbf{x}^i, \mathbf{y}^j) = c(\mathbf{x}^i, \mathbf{y}^j)$ and $\text{LSE}_{k=1}^N(\mathbf{V}_k) = \log \sum_{k=1}^N \exp(\mathbf{V}_k)$ is a log-sum-exp reduction for a vector $\mathbf{V}$. For further details on how the functions, g and s are computed, see along with the SHD and its gradient, see [24, 49].

4.2. Boltzmann machine training

For the RBM, we use the standard Boltzmann protocol of maximizing the log-likelihood function $\mathcal{L}^5$, i.e. the probability of generating vectors belonging to a training set $Y = \{\mathbf{y}\}$:

$$\mathcal{L}(Y, \theta) = \log(p_\theta(Y)), \quad (18)$$

⁵ Equivalent to minimizing an empirical cost $\tilde{D}(p_\theta, \pi) = 1 - \mathcal{L}(Y, \theta)$. The maximization procedure adds an extra negative sign to the update rule.

where θ are the Boltzmann machine model parameters and $p(y)$ is the probability of generating data vectors $y \sim \pi(y)$. For a particular data vector $y \in Y$, we can take the likelihood function $\mathcal{L}$ as our cost function as a function of the model parameters $\theta = \mathcal{W} = \{W_e\}$ [55], which results in the gradient:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial W_e} = \langle v_e \rangle_{\pi} - \langle v_e \rangle_{p_{\theta}} \quad (19)$$

wherein W_e of (4) are the model parameters coupling their respective set of nodes $v_e := \prod_{v_i \in e} v_i$, $\langle v_e \rangle_{\pi}$ and $\langle v_e \rangle_{p_{\theta}}$ are the expectation values of v_e calculated from the data and model distributions respectively where v_e is taken to be a random variable taking values $\{0, 1\}$.

As an example, consider an update to a (visible node) bias term (i.e. a self-loop in figure 2), we have $W_e = b_{v_i}$ and also v_e is simply b_{v_i} since the edge connects only one node. Then the gradient is computed using the expectation value of the bias:

$$\frac{\partial \mathcal{L}}{\partial b_{v_i}} = \langle b_{v_i} \rangle_{\pi} - \langle b_{v_i} \rangle_{p_{\theta}}. \quad (20)$$

In this work, we use vanilla gradient descent as the update rule, but we note we also considered more complex update rules or optimizers such as Adam [46], and we found that this only improved the convergence speed, and not the final accuracy of the model.

The above discussion has no quantum component, as the update rule and the model is completely classical. However, in order to actually compute the first and second order moment terms (using M data and N model-generated bitstrings) in (19), we require a method of generating samples from the RBM. Unlike the Born machine, sample generation from a Boltzmann machine is not a trivial matter. Typical approaches are based on Gibbs sampling, for example k -step contrastive divergence [56]. Here, we use a method called QxSQA, a GPGPU-accelerated simulated quantum annealer based on path-integral Monte Carlo [57]. This simulated quantum annealing has been shown to be useful for sampling Boltzmann-like distributions, and we have shown the ability to use this sampling to train large *quantum* generalized Boltzmann machines (QGBMs) for the purposes of generating synthetic data based on images [58] and financial data [14, 26] in previous research [59].

5. Results

Here we present the numerical results obtained above using the models and training methods detailed above. In particular, we focus of training using the SHD with the Adam optimiser [46] and its analytic gradients for the Born machine, and log-likelihood maximization using QxSQA for training the Boltzmann machine. In appendix A, we revisit alternative training methods. As mentioned above, in the first instance, we also fix both models to only have trainable local parameters for simplicity. For the Born machine, this corresponds to only training single qubit unitaries, with the two qubit gates being unparameterised, and for the Boltzmann machine, this corresponds to training the biases of each node. We force each model to have the same number of trainable parameters in this way. The entanglement structure in states produced in the Born machine is fixed by the problem size, via the lattice topology, and the weights of the RBM are chosen to be random (but fixed) values on each instance. It is difficult to directly compare the connectivity of the models, however we also experimented with randomly pruning the RBM weights to enforce the same number of connections as in the QCBM, but we found this did not affect performance significantly.

As a method of benchmarking the expressive power of each model in a fair way, we use an adversarial discriminator, and judge the performance relative to it. Specifically, we use a random forest discriminator from `scikit-learn` [60] with 1000 estimators. A higher discriminator error implies better performance, with an error of 50% indicates the discriminator can at best guess randomly when presented with a sample as to its origin—whether it came from the real data, or the model. Where error bars are shown, they correspond to the mean and standard deviations of the training over 5 independent runs. As the QCBM scales, the classical simulation becomes a bottleneck and limits the number of runs which can be done.

In summary, we find the Born machine has the capacity to outperform the RBM as the precision of the currency pairs increases. In figure 9, we use data from 2 currency pairs, at 2, 3, 4 and 6 bits of precision. We notice the Born machine outperforms the RBM around 4 bits (measured by a higher discriminator error), and still performs relatively well when run on the QPU. Similar behaviour is observed for 3 currency pairs in figure 11, which uses a precision of 2 and 4 bits, and with 4 pairs in figure 12 for a precision of 2 and 3 bits. In figure 13 we plot the entangling capability (defined by (9)) of the states generated by initial and final circuits learned via training. Curiously, we notice that in the problem instances in which the Born machine outperforms the Boltzmann machine (those with a higher level of precision), the trained circuits produce

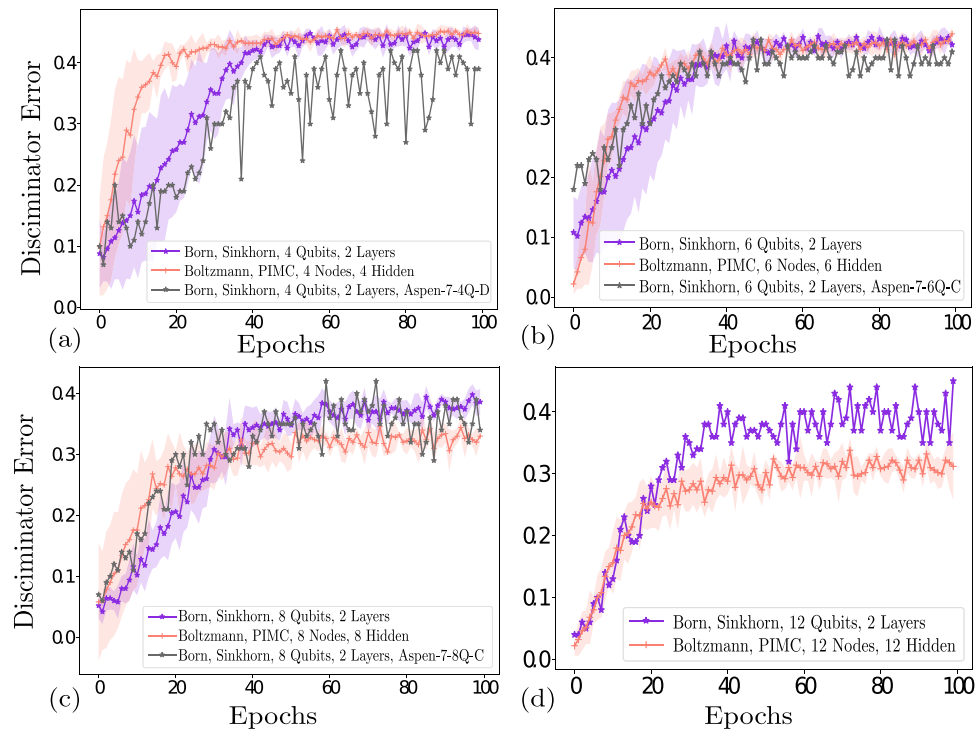


Figure 9. 2 currency pairs (specifically EUR/USD and GBP/USD) at (a) 2 bits, (b) 3 bits, (c) 4 bits and (d) 6 bits of precision. Correspondingly, we use a QCBM of 4, 6, 8 and 12 qubits using the ansätze described above, and an RBM with the same numbers of visible nodes. The hidden units are scaled in each case to match 2 layers of the QCBM. Results when the QCBM is run on sublattices of the AspenTM-7 QPU are shown in grey, whereas the simulated version is given by the purple line, with a noise model discussed in section 5.1.

states which have a higher level of entanglement than those that do not, despite the data being completely classical in nature. This is especially prominent for 2 currency pairs in figure 13(a), in which the training drives the entanglement capability at 2 and 3 bits of precision close to zero (even for increased numbers of layers), but it is significantly higher for 4 and 6 bits of precision, when the Born machine outperforms the RBM, as seen in figure 9. Similar behaviour is seen for 3 currency pairs, but not as evident for 4 pairs. The latter effect is possibly correlated to the similar performance of both models for 4 currency pairs up to 3 bits of precision. The observed behaviour of the entangling capability of the QCBM states is one direction in exploring *why* the model may demonstrate an advantage relative to the RBM and provides possibly the most important question raised by this work for future investigation. Seeking explanations for certain advantages could be beneficial in designing future QML algorithms which can actively exploit such features.

In all the figures presented here, where shown (we typically repeated runs only for small problem instances due to the overheads required to simulate the QCBM), errors bars indicate mean and standard deviations over 5 independent training runs and deviations are due to the stochastic nature of the training procedure. In all figures, where shown, grey lines indicate training runs on the AspenTM QPU.

As a further measure of visually comparing performance of the trained models, we show QQ plots of the marginal output distributions from one of the outperforming QCBM cases (2 pairs at 6 bits of precision) in figure 10. The QCBM clearly produces a better data fit than the RBM when trained (bottom panels), where a perfect fit would be a straight line, shown in figure 10(a) where the data is plotted against itself. In both cases, we use 5070 samples from the QCBM and RBM, to match the size of the dataset.

In appendix B we investigate changing the parameter count in each model for fixed problem sizes. For both models, we find that increasing the model size past those shown in the main text did not have a significant impact on the expressiveness, indicating that saturation has occurred for these parameter numbers. We notice that increasing the number of hidden nodes (layers) for the RBM (QCBM) slightly decreases (increases) convergence speed, but it does not affect the final converged value appreciably. We are also able to somewhat successfully train the largest instance of a Born machine to date in the literature, namely one consisting of 28 qubits on the Rigetti AspenTM-7 chip using the SHD (whose topology is shown in figure 5(e)), and we find it performs surprisingly well. We show the performance of the 28 qubit model versus the a Boltzmann machine with 28 visible nodes, and a suitable number of hidden nodes to match the number of parameters in the Born machine in figure 14. While the performance of the Born machine is significantly less than that of its counterpart, it is clear that the model is learning (despite

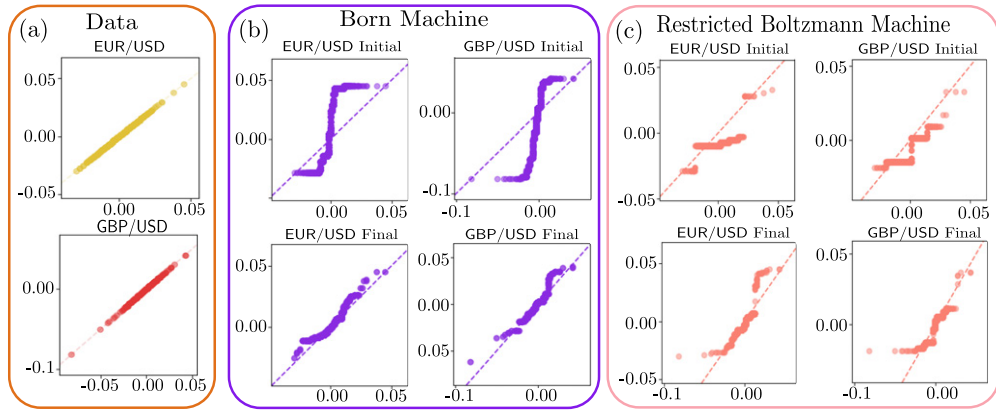


Figure 10. QQ Plots corresponding to figure 9(d) of the marginal distributions of 2 currency pairs (EUR/USD and GBP/USD) at 6 bits of precision. The Born machine distributions (purple) and those generated by the Boltzmann machine in (pink). (a) Shows the QQ plot for the marginal distribution of each currency pair with respect to itself as a benchmark. (b) Born machine initial (top panels) and final (bottom panels) marginal distributions for both pairs and similarly in (c) for the RBM. While not able to completely mimic the data due to the low number of parameters, the Born machine clearly produces a better fit.

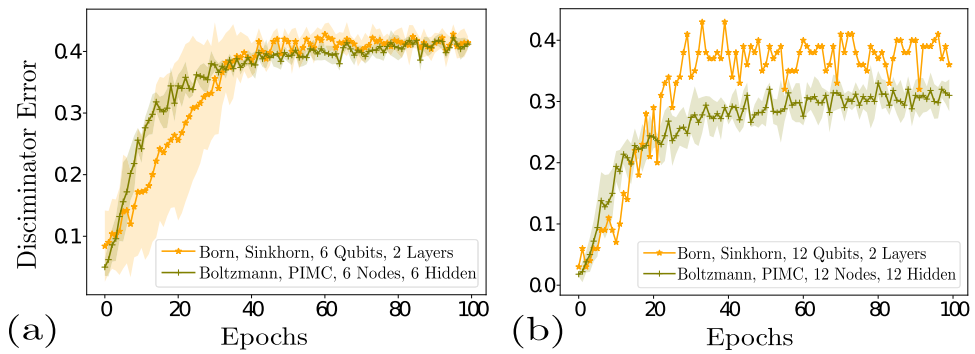


Figure 11. 3 currency pairs at (a) 2 bits and (b) 4 bits of precision, using a QCBM of 6 and 12 qubits and an RBM with the same numbers of visible nodes. All results for QCBM shown here are simulated with the noise model discussed in section 5.1.

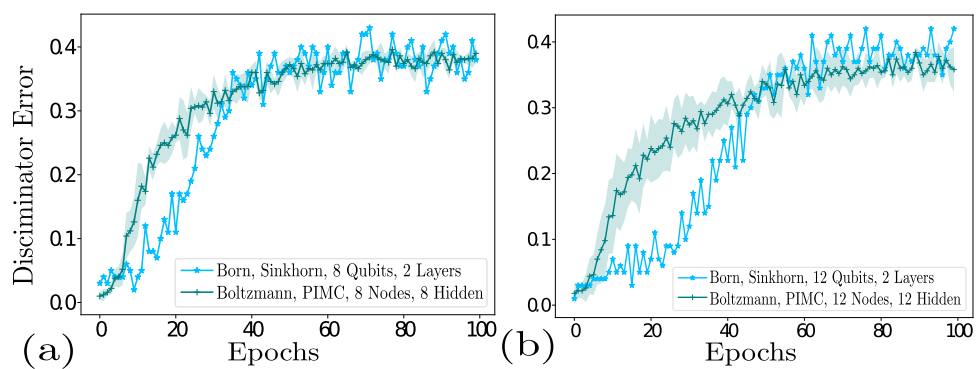


Figure 12. All 4 currency pairs at (a) 2 bits, (b) 3 bits of precision, using a QCBM of 8 and 12 qubits and RBM with the same numbers of visible nodes. We notice the RBM performs similarly here to the QCBM, with a possible advantage for the QCBM observed. The lack of a more pronounced gain in this case is likely due to the smaller bit precision used here. All results for QCBM shown here are simulated with the noise model discussed in section 5.1.

hardware errors), up to a discriminator error of 20%. While this result seems to contradict the previous findings in this work, we emphasize that it does not, since we are not able to simulate the QCBM at this scale in a reasonable amount of time. We would not necessarily expect the Born machine to match the performance of the RBM *on hardware* at this scale for a number of reasons, the most likely cause for diminishing performance is quantum errors in the hardware. However we cannot rule out other factors, such as the ansatz choice. We leave thorough investigation of improving hardware performance to future

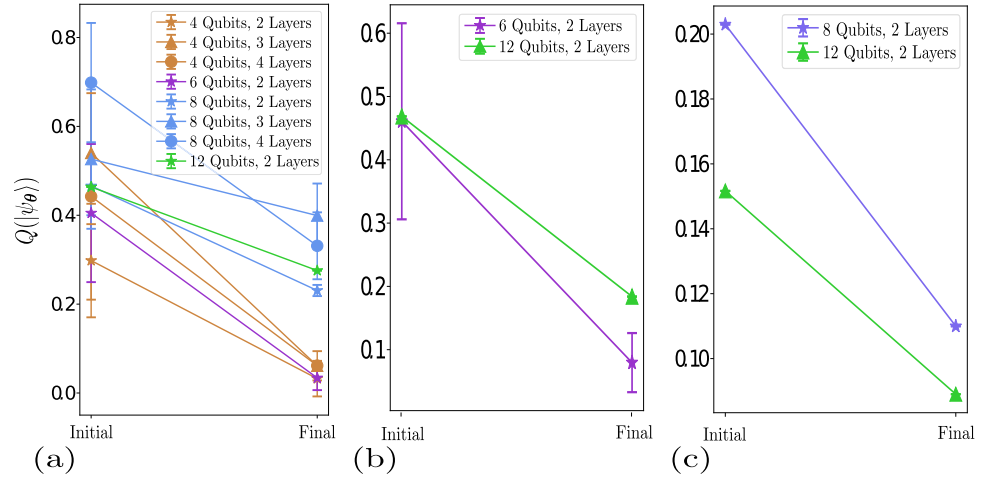


Figure 13. Meyer–Wallach entangling capability (9) for a random choice of parameters (initial) and the trained parameters (final) in the same circuit. The above results are generated by simulating the corresponding circuits. Error bars represent mean and standard deviation over 5 independent training runs, where they are shown. The circuit ansätze used are those above in figure 7 closely matching the corresponding chip topology. In each panel we see the entanglement in the final states trained on (a) 2 currency pairs at 2, 3, 4, 6 bits of precision, (b) 3 currency pairs at 2 and 4 bits of precision and (c) 4 currency pairs at 2 and 3 bits of precision.

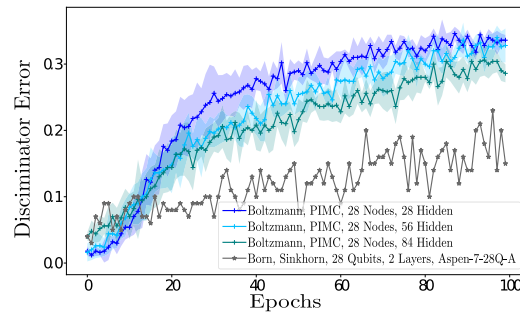


Figure 14. Random forest discriminator during training for a problem size of 4 currency pairs at 7 bits of precision, using 28 visible nodes in the Boltzmann machine and 28 qubits in the Born machine. The 28 qubit Born machine is run exclusively on the AspenTM-7-28Q-A chip using 2 layers of the hardware efficient ansatz similar to those shown in figure 7.

work, perhaps by including error mitigation [61] to reduce errors, thorough error modelling and parametric compilation and active qubit reset [15, 62] to improve running time and other techniques.

5.1. Experimental details

Here we elaborate on some experimental parameters we used to produce the above results. Firstly, in all simulated versions of the QCBM, the noise model used is that attached to the `noisy-qvm` version of a quantum device, which is a simple noise model including readout errors and standard T_1 and T_2 times. Details can be found in the source code of PyQuil[®] [62]. When run on the QPU, the experimental parameters are exactly the same as in the simulated case. We implement the ansätze exactly as discussed above (which in itself is very close to hardware native) and leave any further compilation to the Rigetti compiler. For training the model using the SHD, we choose $\epsilon = 0.5$ which we determined by a basic hyperparameter search using small problem sizes. We also used the Adam optimiser with standard parameters [46] and an initial learning rate of $\eta = 0.05$. In all cases, we used 250 samples from both the data and the QCBM/RBM in all circuits for simplicity. A question for future work is the necessary sample sizes for successful training.

To generate RBM samples, we used the following parameters whose definitions can be found in [57]: $\Gamma_0 = 3$, $\Gamma_\tau = 1 \times 10^{-20}$, $N_{\text{sweeps}} = 1$, $PT = 0.1$, $N_{\text{anneals}} = 250$, $T_{\text{eff}} = 1$. Using QxSQA to perform approximate sampling of the RBM is a source of error in the model, but in previous work [57] the method was shown to provide sufficiently good approximation to the required Boltzmann distribution to be considered near exact, for a large range of model sizes, even orders of magnitude larger than those considered here.

We did not investigate different embedding methods to match the currency variables to the hardware connectivity in the QCBM, however it has been shown [14] that this can contribute to the performance. Here we chose a simple one to one correspondence between variables and qubit indices in the chip, and leave further investigation of this point to future work.

6. Discussion

In conclusion, we investigate and compare two different models when trained on a real-world financial dataset consisting of currency pairs at varying levels of precision. We chose a completely classical model in the RBM, and put it up against a completely quantum model in the form of a QCBM, in order to compare their relevant expressive powers, and supplement recent related work in this direction [14, 27]. As a benchmark of fairness, we fixed the models to have the same numbers of trainable parameters and found that the simulated Born machine always performed at least as well as the RBM, and in several cases outperformed it, measured relative to the accuracy of an adversarial discriminator. To complement this finding, we investigated the entangling capability of the circuits learned by the QCBM, and found a rough correlation between training towards higher levels of entanglement, and outperforming the classical model.

From this work, there are many possible avenues for exploration. The first, is improving the Born machine training speed by, for example, leveraging GPU accelerated computation of the cost functions, and also incorporating techniques to improve running time and execution on the QPU. Furthermore, to improve performance, one could consider variable structure ansätze [39, 63] or quantum-specific optimizers [64–66] for the model and training. An alternative direction, is to enlarge the suite of classical model comparison to compare the Born machine to, in order to solidify any perceived advantage and extending the model into mixed states to potentially increase the expressive power [22]. Alternatively, one could investigate methods to divide the classical–quantum resources in the learning procedure [67].

Acknowledgments

We thank Alexei Kondratyev for useful discussions and comments, as well as the Rigetti Computing AspenTM-7 hardware team for access necessary for the experiments in this research. This work was supported by the Engineering and Physical Sciences Research Council (Grant No. EP/L01503X/1), EPSRC Centre for Doctoral Training in Pervasive Parallelism at the University of Edinburgh, School of Informatics and Entrapping Machines, (Grant No. FA9550-17-1-0055).

Appendix A. Alternative training methods

Here we provide numerical results illustrating the training of the Born machine using some alternative methods and cost functions, for small numbers of qubits.

A.1. Maximum mean discrepancy

The first alternative method is derived by using a different cost function, the so-called MMD. Like optimal transport, this defines a metric on the space of probability distributions, and from which, an efficient-to-compute method of comparison can be defined [68, 69]:

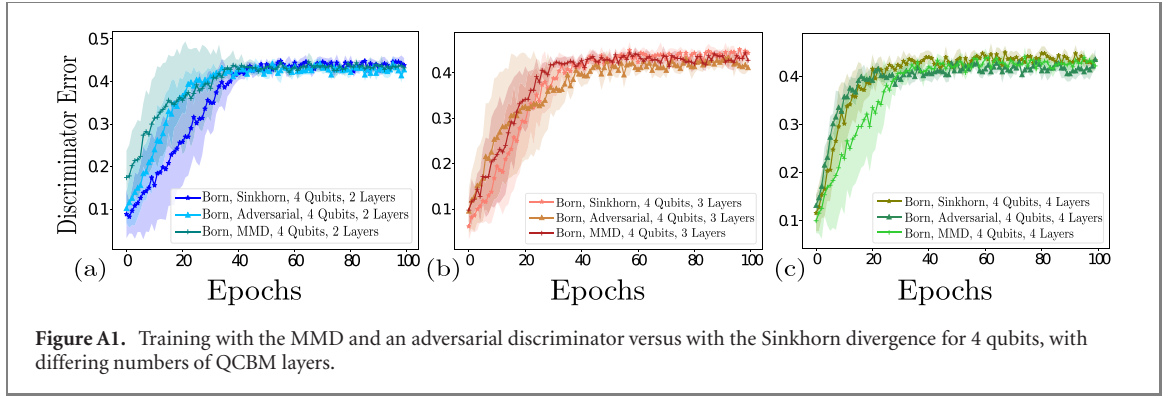
$$D(p_\theta, \pi) := \mathcal{L}_{\text{MMD}} := \mathbb{E}_{\substack{\mathbf{x} \sim p_\theta \\ \mathbf{y} \sim \pi}} (\kappa(\mathbf{x}, \mathbf{y})) + \mathbb{E}_{\substack{\mathbf{x} \sim \pi \\ \mathbf{y} \sim \pi}} (\kappa(\mathbf{x}, \mathbf{y})) - 2\mathbb{E}_{\substack{\mathbf{x} \sim p_\theta \\ \mathbf{y} \sim \pi}} (\kappa(\mathbf{x}, \mathbf{y})) \quad (\text{A.1})$$

This cost function was originally utilized for hypothesis testing [69], but has since found use in training generative models. In particular, it enabled the first approach to train a QCBM [21, 24] in a differentiable way.

The function κ is a *kernel* function, which enables a means of comparison on the support spaces, $\mathcal{X}, \mathcal{Y}$. For this work, we choose the common Gaussian mixture kernel [21] for the MMD, which is universal, and hence enables the MMD to act as a faithful method of distribution comparison:

$$\kappa_G(\mathbf{x}, \mathbf{y}) := \frac{1}{c} \sum_{i=1}^c \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|_2^2}{2\sigma_i}\right) \quad (\text{A.2})$$

The parameters, σ_i , are *bandwidths* which determine the scale at which the samples are compared, and $\|\cdot\|_2$ is the ℓ_2 norm. Here we choose $\sigma = [0.25, 10, 1000]$, as in [21]. Typically, the kernel is a classical function, but quantum kernels can also be considered here [24, 70–72].



A.2. Adversarial discriminator

The second method we can choose to use is to not only use a discriminator as a benchmark, but also to train the model relative to it. As in the above cases, this is a gradient based approach, with the analytic gradient taken by differentiating the discriminator loss.

Adversarial training has become a popular and powerful way to train neural networks, originating with GANs [17]. GANs are composed of two machine learning components, a discriminator, $\mathcal{D}$, which attempts to predict if a sample $\mathbf{x}$ is from a data distribution or rather has been generated by a generator network $\mathcal{G}$ (in our notation, the generator network samples from a probability distribution, p_θ and is either a Born machine or a Boltzmann machine). Generalizations of the GAN into the quantum domain have also been considered [7, 28, 73–75]. The generator attempts to minimize the following loss:

$$\mathcal{L}_G := \mathbb{E} [\log (1 - \mathcal{D}(\mathbf{x}))] \approx \frac{1}{N} \sum_{i=1}^N [\log (1 - \mathcal{D}(\mathbf{x}_i))] \quad (\text{A.3})$$

where $\mathcal{D}(\mathbf{x})$ is the probability that a discriminator, $\mathcal{D}$, guesses that $\mathbf{x}$ is from the real data set. The approximation to the expectation value is taken over N generated samples in practice. In order to train the generator with respect to this cost function (taken with respect to a specific discriminator, $\mathcal{D}$), gradient descent can be used to minimize (A.3), with the gradient given by:

$$\nabla_\theta \mathbb{E} [\log (1 - \mathcal{D}(\mathbf{x}_i))] = \nabla_\theta \sum_{\mathbf{x}} p_\theta(\mathbf{x}) [\log (1 - \mathcal{D}(\mathbf{x}_i))] = \sum_{\mathbf{x}} \nabla_\theta p_\theta(\mathbf{x}) [\log (1 - \mathcal{D}(\mathbf{x}_i))] \quad (\text{A.4})$$

If we again assume the generator network is a Born machine, composed of quantum gates of the form $U(\theta) = \exp(i\theta/2\Sigma)$, then using the parameter shift rule as for the Sinkhorn divergence above (14), we get:

$$\nabla_\theta \mathcal{L}_G = \frac{1}{2} \sum_{\mathbf{x}} p_{\theta^+} [\log (1 - \mathcal{D}(\mathbf{x}_i))] - p_{\theta^-} [\log (1 - \mathcal{D}(\mathbf{x}_i))] \quad (\text{A.5})$$

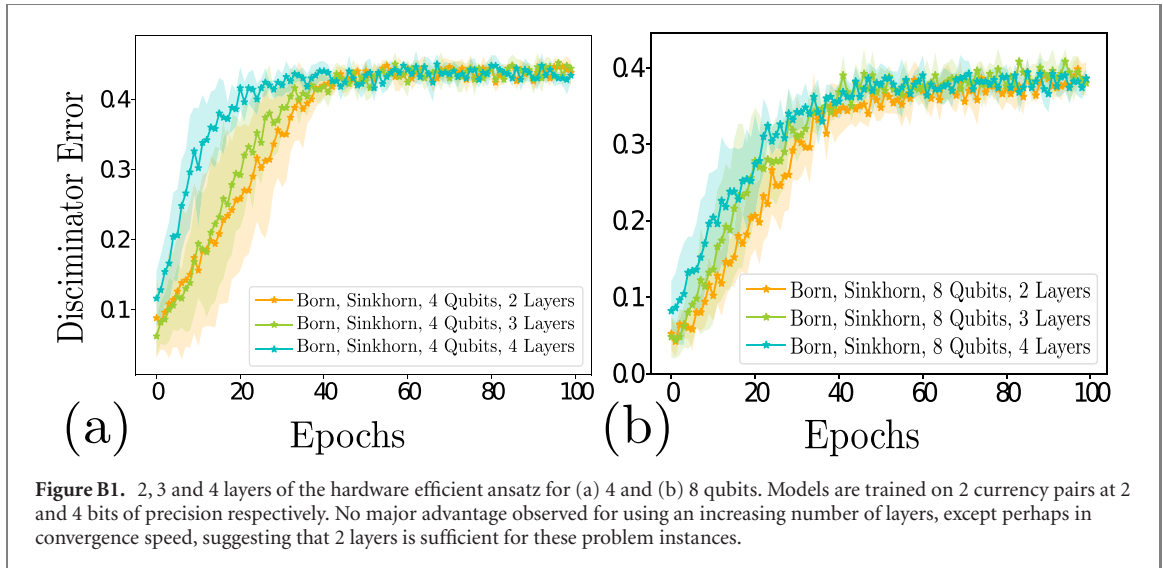
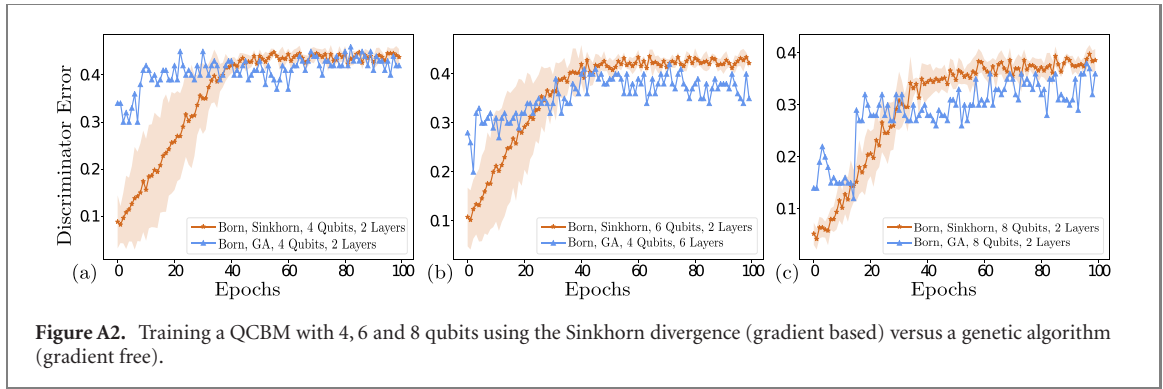
These expectation values can be evaluated by sampling from the parameter shifted circuit distributions, $p_{\theta^\pm}$ as usual. Correspondingly, while a generator is trying to minimize the above cost, (A.3), the discriminator can also be trained for a number of sub-steps to become better at identifying false samples. This can be done by using gradient *ascent* to maximize the following cost:

$$\mathcal{L}_D \approx \frac{1}{M} \sum_{j=1}^M \log \mathcal{D}(\mathbf{y}_j) + \frac{1}{N} \sum_{i=1}^N \log (1 - \mathcal{D}(\mathbf{x}_i)) \quad (\text{A.6})$$

where the latter term is the same as in (A.4), and the former represents the probability that $\mathcal{D}$ is able to correctly identify true data samples, $\mathbf{y} \sim \pi$. The gradient of (A.6) can be computed similarly (figure A1).

In this work, we implement the training laid out in this section with two slight variations to note:

- (i) We used a slightly revised version of equation (A.5) which dropped the 0.5 and log components (simply used $1 - \mathcal{D}$ in both terms). As we were still using Adam as the update optimizer, we believe that asserting this overall should not pose any major impact to performance. Moreover, the adversarial approach was still slower compared to the Sinkhorn divergence, and therefore did not garner increased focus in this work.
- (ii) As the modelling problem in this work was extremely small, we choose to simply re-train a new discriminator model from scratch at every generative model training iteration, with corresponding



test set error of the discriminator being recorded and used as a primary metric in this work. Similarly, a different discriminator model was used for calculating model parameter updates every training iteration while using adversarial training.

A.3. Genetic algorithm

Finally, we use a gradient free approach in a genetic algorithm, since this was also used to train a QCBM on this same dataset [14]. One could also choose one of the many gradient free optimisers from `scikit-learn`, as has been also done for Born machines [20]. A simplified version of a genetic algorithm was implemented in [14] due to the low number of parameters in a 12 qubit Born machine. We found that this method was significantly slower than the gradient based methods we discuss above (figure A2).

Appendix B. Alternative model structures

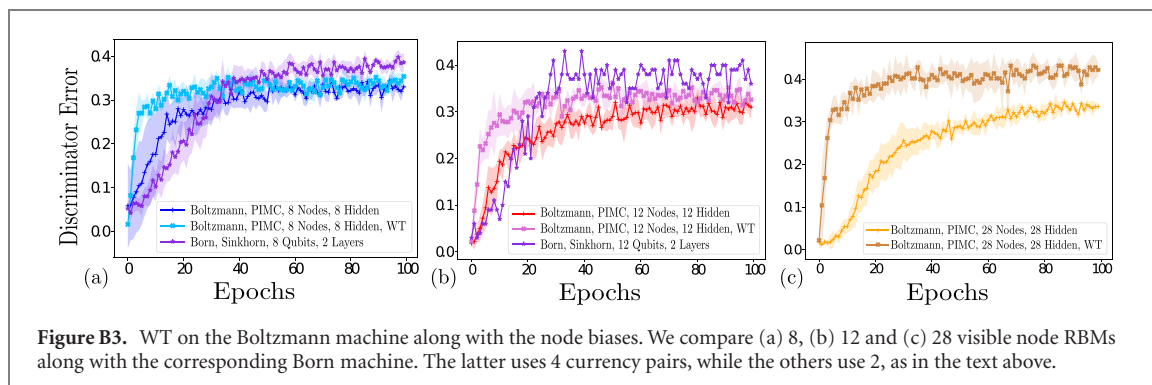
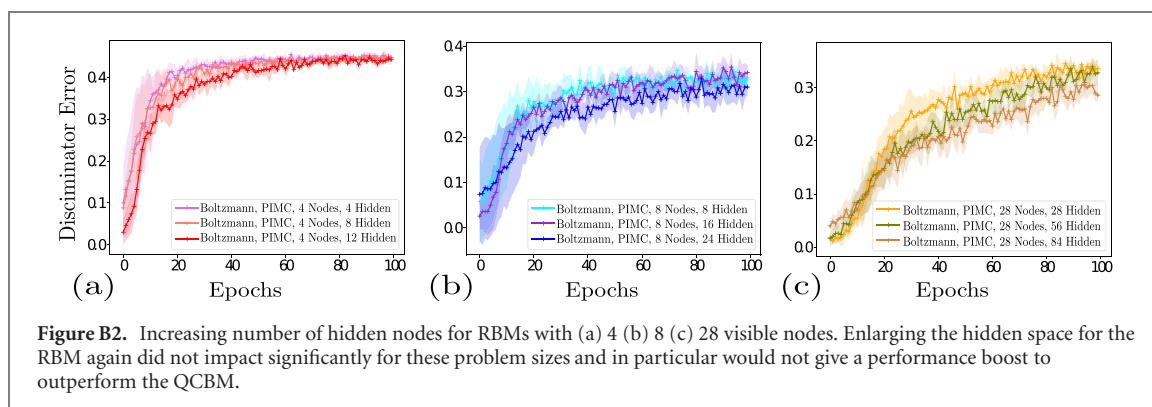
Here we showcase the effect of using alternative model structures for the QCBM and the RBM.

B.1. Differing numbers of Born machine layers

For completeness, in figure B1, we show the effect of alternating the number of layers of the hardware efficient Ansatz, shown in figure 7 for 4 and 8 qubits. In particular, we notice that increasing the number of layers does not have a significant impact, at least at these scales, except perhaps in convergence speed of the training. It is likely however, that at larger scales, increased parameter numbers would be required to improve performance.

B.2. Differing numbers of Boltzmann hidden nodes

We also demonstrate the effect of changing the number of hidden nodes in the Boltzmann machine in figure B2, where we have 4, 8 and 28 visible nodes. Again, we observe that an increasing number of hidden nodes (and by extension, number of parameters) does not substantially affect the performance of the



model, in fact it can hinder it, at least when training only biases of the Boltzmann machine. In particular, it does not substantially alter the final accuracy achieved by the model. We noticed similar behavior when also training the weights of the Boltzmann machine.

B.3. Weight training of Boltzmann machine

Finally, we compare the effect of weight training (WT) of the Boltzmann machine to training the bias terms alone. For the problem instances where the Boltzmann machine was able to converge to the best discriminator accuracy (i.e. in the small problem instances), we find training the weights has the effect of increasing convergence speed, and also increased accuracy where training the biases only was insufficient to achieve high discriminator error. Interestingly, we note that the Born machine still outperforms the 8 and 12 visible node RBMs, even when the weights are also trained, and this does not seem to majorly affect the performance. However, training the weights does make a large difference for 28 nodes, as seen in figure B3(c), so again further investigation is needed in future work of this phenomenon.

ORCID iDs

Brian Coyle  <https://orcid.org/0000-0002-3436-8458>

References

- [1] Preskill J 2018 Quantum computing in the NISQ era and beyond *Quantum* **2** 79
- [2] Frank A et al 2019 Quantum supremacy using a programmable superconducting processor *Nature* **574** 505–10
- [3] McArdle S, Endo S, Aspuru-Guzik A, Benjamin S C and Yuan X 2020 Quantum computational chemistry *Rev. Mod. Phys.* **92** 015003
- [4] Frank A et al 2020 Hartree–Fock on a superconducting qubit quantum computer (arXiv:2004.04174)
- [5] Farhi E, Goldstone J and Gutmann S 2014 A quantum approximate optimization algorithm (arXiv:1411.4028)
- [6] Frank A et al 2020 Quantum approximate optimization of non-planar graph problems on a planar superconducting processor (arXiv:2004.04197)
- [7] Zoufal C, Lucchi A and Woerner S 2019 Quantum generative adversarial networks for learning and loading random distributions *npj Quantum Inf.* **5** 103
- [8] Ramos-Calderer S, Pérez-Salinas A, García-Martín D, Bravo-Prieto C, Cortada J, Planagumà J and Latorre J I 2019 Quantum unary approach to option pricing (arXiv:1912.01618)
- [9] Rebertrost P, Gupt B and Bromley T R 2018 Quantum computational finance: Monte Carlo pricing of financial derivatives *Phys. Rev. A* **98** 022321

- [10] Mugel S, Kuchkovsky C, Sanchez E, Fernandez-Lorenzo S, Luis-Hita J, Lizaso E and Roman O 2020 Dynamic portfolio optimization with real datasets using quantum processors and quantum-inspired tensor networks (arXiv:2007.00017)
- [11] Shor P W 1997 Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer *SIAM J. Comput.* **26** 1484–509
- [12] Harrow A W, Hassidim A and Lloyd S 2009 Quantum algorithm for linear systems of equations *Phys. Rev. Lett.* **103** 150502
- [13] Ronnow T F, Wang Z, Job J, Boixo S, Isakov S V, Wecker D, Martinis J M, Lidar D A and Troyer M 2014 Defining and detecting quantum speedup *Science* **345** 420–4
- [14] Kondratyev A 2020 Non-differentiable learning of quantum circuit Born machine with genetic algorithm (Elsevier BV) available SSRN: <https://ssrn.com/abstract=3569226>
- [15] Karalekas P J, Tezak N A, Peterson E C, Ryan C A, da Silva M P and Smith R S 2020 A quantum-classical cloud platform optimized for variational hybrid algorithms *Quantum Sci. Technol.* **5** 024003
- [16] Mohamed S and Lakshminarayanan B 2016 Learning in implicit generative models (arXiv:1610.03483 [cs, stat])
- [17] Goodfellow I J, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A and Bengio Y 2014 Generative adversarial networks (arXiv:1406.2661 [cs, stat])
- [18] Nathan W, Kapoor A, Granade C and Svore K M 2015 Quantum inspired training for Boltzmann machines (arXiv:1507.02642 [quant-ph])
- [19] Cheng S, Chen J and Wang L 2018 Information perspective to probabilistic modeling: Boltzmann machines versus Born machines *Entropy* **20** 583
- [20] Benedetti M, Garcia-Pintos D, Perdomo O, Leyton-Ortega V, Nam Y and Perdomo-Ortiz A 2019 A generative modeling approach for benchmarking and training shallow quantum circuits *npj Quantum Inf.* **5** 45
- [21] Liu J-G and Wang L 2018 Differentiable learning of quantum circuit Born machines *Phys. Rev. A* **98** 0 62324
- [22] Verdon G, Marks J, Nanda S, Leichenauer S and Hidary J 2019 Quantum Hamiltonian-based models and the variational quantum thermalizer algorithm (arXiv:1910.02071)
- [23] Du Y, Hsieh M-H, Liu T and Tao D 2018 The expressive power of parameterized quantum circuits (arXiv:1810.11922 [quant-ph])
- [24] Coyle B, Mills D, Danos V and Kashefi E 2020 The Born supremacy: quantum advantage and training of an Ising Born machine *npj Quantum Inf.* **6** 60
- [25] Tangpanitanon J, Thanasilp S, Dangniam N, Lemonde M-A and Angelakis D G 2020 Expressibility and trainability of parameterized analog quantum systems for machine learning applications (arXiv:2005.11222)
- [26] Kondratyev A and Schwarz C 2019 The market generator available SSRN: <https://ssrn.com/abstract=3384948>
- [27] Alcazar J, Leyton-Ortega V and Perdomo-Ortiz A 2020 Classical versus quantum models in machine learning: insights from a finance application *Mach. Learn.: Sci. Technol.* **1** 035003
- [28] Romero J and Aspuru-Guzik A 2019 Variational quantum generators: generative adversarial quantum machine learning for continuous distributions (arXiv:1901.00848 [quant-ph])
- [29] Verdon G, Pye J and Broughton M 2018 A universal training algorithm for quantum deep learning (arXiv:1806.09729)
- [30] Goodfellow I, Bengio Y and Courville A 2016 *Deep Learning* (Cambridge, MA: MIT Press)
- [31] Amin M H, Andriyash E, Rolfe J, Kulchitsky B and Melko R 2018 Quantum Boltzmann machine *Phys. Rev. X* **8** 0 21050
- [32] Kieferova M and Nathan W 2017 Tomography and generative data modeling via quantum Boltzmann training *Phys. Rev. A* **96** 062327
- [33] Song H J, Song T, He Q K, Liu Y and Zhou D L 2019 Geometry and symmetry in the quantum Boltzmann machine *Phys. Rev. A* **99** 042307
- [34] Nathan W and Leonard W 2019 Generative training of quantum Boltzmann machines with hidden units (arXiv:1905.09902)
- [35] Verdon G, Broughton M and Jacob B 2017 A quantum algorithm to train neural networks using low-depth circuits (arXiv:1712.05304)
- [36] Schuld M, Bocharov A, Svore K M and Nathan W 2020 Circuit-centric quantum classifiers *Phys. Rev. A* **101** 032308
- [37] McClean J R, Boixo S, Smelyanskiy V N, Ryan B and Neven H 2018 Barren plateaus in quantum neural network training landscapes *Nat. Commun.* **9** 4812
- [38] Cerezo M, Sone A, Tyler V, Cincio L and Coles P J 2020 Cost-function-dependent barren plateaus in shallow quantum neural networks (arXiv:2001.00550 [quant-ph])
- [39] Cerezo M, Sharma K, Arrasmith A and Coles P J 2020 Variational quantum state eigensolver (arXiv:2004.01372 [quant-ph])
- [40] Grant E, Wossnig L, Ostaszewski M and Benedetti M 2019 An initialization strategy for addressing barren plateaus in parametrized quantum circuits *Quantum* **3** 214
- [41] Huang H-Y, Bharti K and Rebentrost P 2019 Near-term quantum algorithms for linear systems of equations (arXiv:1909.07344 [quant-ph])
- [42] Meyer D A and Wallach N R 2002 Global entanglement in multiparticle systems *J. Math. Phys.* **43** 4273–8
- [43] Sim S, Johnson P D and Aspuru-Guzik A 2019 Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms *Adv. Quantum Technol.* **2** 1900070
- [44] Hubregtsen T, Pichlmeier J and Bertels K 2020 Evaluation of parameterized quantum circuits: on the design, and the relation between classification accuracy, expressibility and entangling capability (arXiv:2003.09887 [quant-ph])
- [45] Brennen G K 2003 An observable measure of entanglement for pure states of multi-qubit systems (arXiv:quant-ph/0305094)
- [46] Kingma D P and Jimmy B 2015 Adam: a method for stochastic optimization *3rd Int. Conf. Learn. Represent. (ICLR) 2015* (San Diego, CA, USA 7–9 May 2015) ed Y Bengio and Y LeCun Conf. Track Proc.
- [47] Ramdas A, Garcia N and Cuturi M 2015 On Wasserstein two sample testing and related families of nonparametric tests (arXiv:1509.02237 [math, stat])
- [48] Genevay A, Gabriel P and Cuturi M 2018 Learning generative models with Sinkhorn divergences *Proc. 21st Int. Conf. Artif. Intell. Stat. (Proceedings of Machine Learning Research vol 84)* (Playa Blanca, Lanzarote, Canary Islands) ed A Storkey and F Perez-Cruz pp 1608–17 PMLR
- [49] Jean F, Séjourné T, Vialard F-X, Amari S, Troune A and Gabriel P 2019 Interpolating between optimal transport and MMD using Sinkhorn divergences *Proc. Mach. Learn. Res. (Proceedings of Machine Learning Research vol 89)* ed K Chaudhuri and M Sugiyama pp 2681–90 PMLR
- [50] Villani C 2009 *Optimal Transport: Old and New (Grundlehren der mathematischen Wissenschaften)* (Berlin: Springer)
- [51] Kullback S and Leibler R A 1951 On information and sufficiency *Ann. Math. Stat.* **22** 79–86
- [52] Mitarai K, Negoro M, Kitagawa M and Fujii K 2018 Quantum circuit learning *Phys. Rev. A* **98** 0 32309

- [53] Schuld M, Bergholm V, Gogolin C, Izaac J and Nathan K 2019 Evaluating analytic gradients on quantum hardware *Phys. Rev. A* **99** 032331
- [54] Sinkhorn R 1964 A relationship between arbitrary positive matrices and doubly stochastic matrices *Ann. Math. Stat.* **35** 876–9
- [55] Fischer A and Igel C 2014 Training restricted Boltzmann machines: an introduction *Pattern Recognit.* **47** 25–39
- [56] Hinton G E 2012 A practical guide to training restricted Boltzmann machines *Neural Networks: Tricks of the Trade* vol 7700 ed G Montavon, G B Orr and K-R Müller (Berlin: Springer) pp 599–619
- [57] Padilha D, Weinstock S and Hodson M 2019 QxSQA: GPGPU-accelerated simulated quantum annealer within a non-linear optimization and Boltzmann sampling framework 2019 *IEEE High Performance Extreme Computing Conf. (HPEC)* pp 1–8
- [58] LeCun Y, Bottou L, Bengio Y and Haffner P 1998 Gradient-based learning applied to document recognition *Proc. IEEE* **86** 2278–324
- [59] Henderson M P and Chan Jin Le J 2019 Generation of industry-relevant synthetic data using simulated quantum annealing-trained Boltzmann machines *QTML - Quantum Tech. Mach. Learn.* (Daejeon, South Korea)
- [60] Pedregosa F et al 2011 Scikit-learn: machine learning in Python *J. Mach. Learn. Res.* **12** 2825–30
- [61] Hamilton K E and Pooser R C 2019 Error-mitigated data-driven circuit learning on noisy quantum hardware (arXiv:1911.13289 [quant-ph])
- [62] Smith R S, Curtis M J and Zeng W J 2016 A practical quantum instruction set architecture (arXiv:1608.03355 [quant-ph])
- [63] Cincio L, Subaşı Y, Sornborger A T and Coles P J 2018 Learning the quantum algorithm for state overlap *New J. Phys.* **20** 113022
- [64] Kübler J M, Arrasmith A, Cincio L and Coles P J 2020 An adaptive optimizer for measurement-frugal variational algorithms *Quantum* **4** 263
- [65] Arrasmith A, Cincio L, Somma R D and Coles P J 2020 Operator sampling for shot-frugal optimization in variational algorithms (arXiv:2004.06252 [quant-ph])
- [66] Lavrijsen W, Tudor A, Müller J, Iancu C and de Jong W 2020 Classical optimizers for noisy intermediate-scale quantum devices (arXiv:2004.03004 [quant-ph])
- [67] Painsi M and Kalev A 2019 An approximate description of quantum states (arXiv:1910.10543 [quant-ph])
- [68] Borgwardt K M, Gretton A, Rasch M J, Kriegel H-P, Schölkopf B and Smola A J 2006 Integrating structured biological data by Kernel maximum mean discrepancy *Bioinformatics* **22** e49–57
- [69] Gretton A, Borgwardt K M, Rasch M, Schölkopf B and Smola A J 2007 A Kernel method for the two-sample-problem *Advances in Neural Information Processing Systems* vol 19 ed B Schölkopf, J C Platt and T Hoffman (Cambridge, MA: MIT Press) pp 513–20
- [70] Kübler J M, Muandet K and Schölkopf B 2019 Quantum mean embedding of probability distributions *Phys. Rev. Res.* **1** 033159
- [71] Schuld M and Nathan K 2019 Quantum machine learning in feature Hilbert spaces *Phys. Rev. Lett.* **122** 040504
- [72] Havlíček V, Córcoles A D, Temme K, Harrow A W, Kandala A, Chow J M and Gambetta J M 2019 Supervised learning with quantum-enhanced feature spaces *Nature* **567** 209–12
- [73] Lloyd S and Weedbrook C 2018 Quantum generative adversarial learning *Phys. Rev. Lett.* **121** 040502
- [74] Dallaire-Demers P-L and Nathan K 2018 Quantum generative adversarial networks *Phys. Rev. A* **98** 012324
- [75] Anand A, Romero J, Degroote M and Aspuru-Guzik A 2020 Experimental demonstration of a quantum generative adversarial network for continuous distributions (arXiv:2006.01976 [quant-ph])